



Data Structures

Lecture 6: Sorting Continued

Prof. Wang Haiyan
School of Computer Science
wanghy@njupt.edu.cn

Today's Topics

Last lecture we looked at three sorting algorithms and saw that the best comparison-based sort works in $O(n \log n)$ time

Today:

- Quicksort
- Shellsort
- Stable sorts
- Rules of thumb for sorting
- Selection problems

Reflections on Merge Sort

- Merge sort gave us an elegant sorting algorithm with $O(n \log n)$ worst case complexity
- We've also seen that sorting by comparing elements is $\Omega(n \log n)$

Q: Should we leave it there? Is there any reason to try and improve on merge sort? What weaknesses of merge sort might there be?

Reflections on Merge Sort

- Merge sort gave us an elegant sorting algorithm with $O(n \log n)$ worst case complexity
- We've also seen that sorting by comparing elements is $\Omega(n \log n)$

Q: Should we leave it there? Is there any reason to try and improve on merge sort? What weaknesses of merge sort might there be?

A: Let's think about the strengths and weaknesses of merge sort...

Some strengths and weaknesses of merge sort

Strengths:

- Easy to describe, understand and implement
- Can be easily parallelised: at any level of recursion, each half list could be passed to a separate processor/computer
- Worst case = average case = best case = $O(n \log n)$ → totally predictable and dependable

Some strengths and weaknesses of merge sort

Strengths:

- Easy to describe, understand and implement
- Can be easily parallelised: at any level of recursion, each half list could be passed to a separate processor/computer
- Worst case = average case = best case = $O(n \log n)$ → totally predictable and dependable

Weaknesses:

- Relatively high constant factor involved (which is why it is beaten by insertion sort for small inputs) → there may be other $O(n \log n)$ sorting algorithms with lower leading constants – in practice these will be faster
- Think about space efficiency...

Space Complexity of Merge Sort

Recall the pseudocode for merge sort:

Function *MergeSort*($A[1..n]$: array **of** Element)

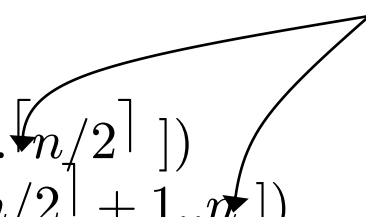
```
1: if  $n = 1$  then  
2:   return  $A$   
3: else  
4:    $A_1 = \text{MergeSort}(A[1..\lceil n/2 \rceil])$   
5:    $A_2 = \text{MergeSort}(A[\lceil n/2 \rceil + 1..n])$   
6:   return  $\text{merge}(A_1, A_2)$   
7: end if
```

Space Complexity of Merge Sort

Recall the pseudocode for merge sort:

Function *MergeSort*($A[1..n]$: array of Element)

```
1: if  $n = 1$  then  
2:   return  $A$   
3: else  
4:    $A_1 = \text{MergeSort}(A[1..\lfloor n/2 \rfloor])$   
5:    $A_2 = \text{MergeSort}(A[\lceil n/2 \rceil + 1..n])$   
6:   return  $\text{merge}(A_1, A_2)$   
7: end if
```



The input sequence can be split in a space efficient manner, just pass index of start and end of each part and keep one copy of input array

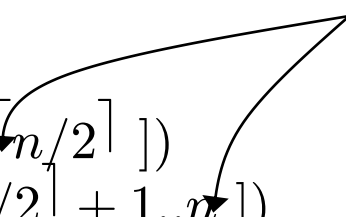
Space Complexity of Merge Sort

Recall the pseudocode for merge sort:

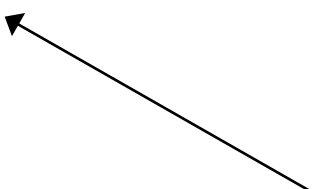
Function *MergeSort*($A[1..n]$: array of Element)

```
1: if  $n = 1$  then  
2:   return  $A$   
3: else  
4:    $A_1 = \text{MergeSort}(A[1..\lfloor n/2 \rfloor])$   
5:    $A_2 = \text{MergeSort}(A[\lceil n/2 \rceil + 1..n])$   
6:   return merge( $A_1, A_2$ )  
7: end if
```

The input sequence can be split in a space efficient manner, just pass index of start and end of each part and keep one copy of input array

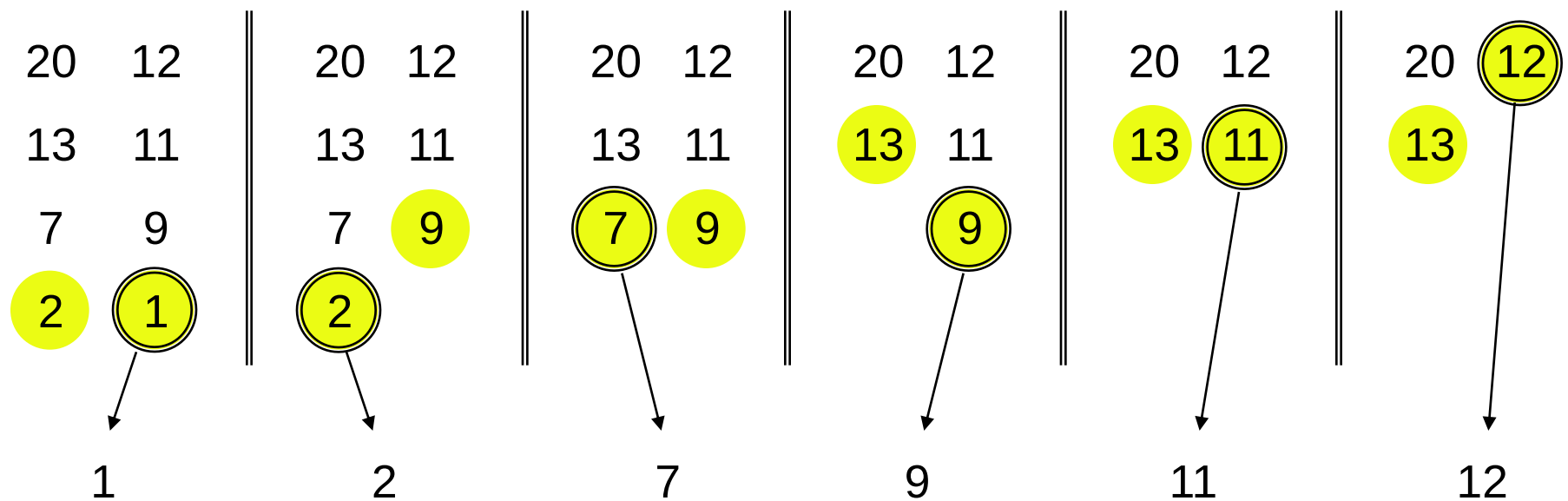


Q: How much extra space do we need to merge two sorted arrays together?



Space Complexity of Merge Sort – Arrays

Recall the merging process:

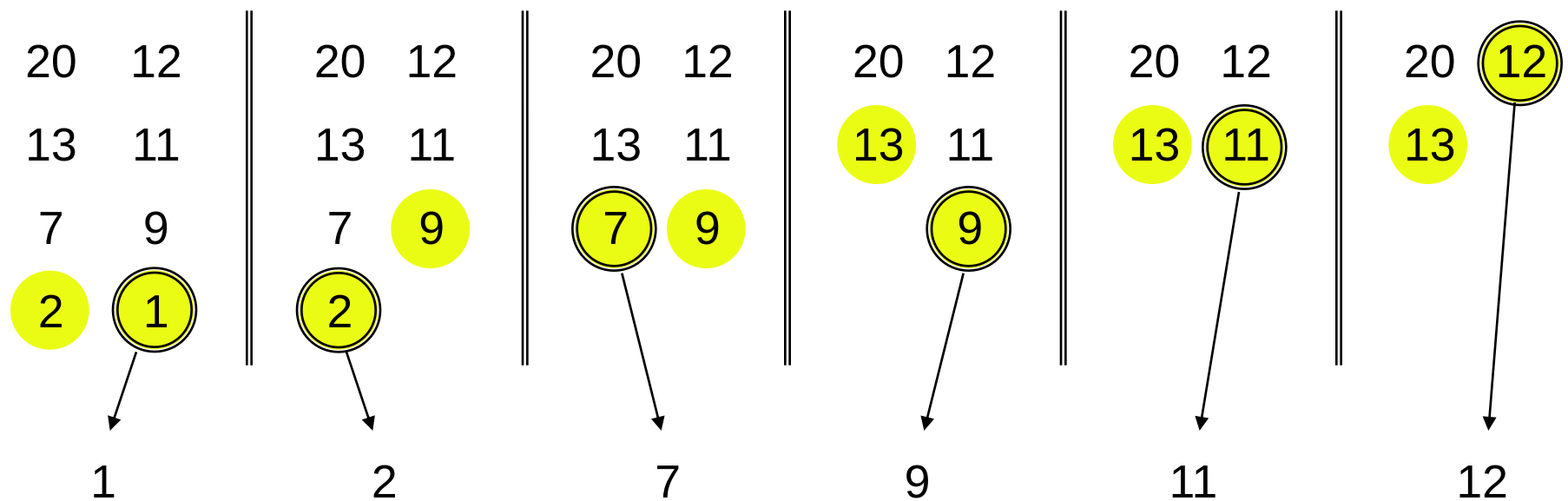


The two sorted lists can initially be stored in either half of the original array

Q: Do we need extra space to merge these two halves together?

Space Complexity of Merge Sort – Arrays

Recall the merging process:



The two sorted lists can initially be stored in either half of the original array

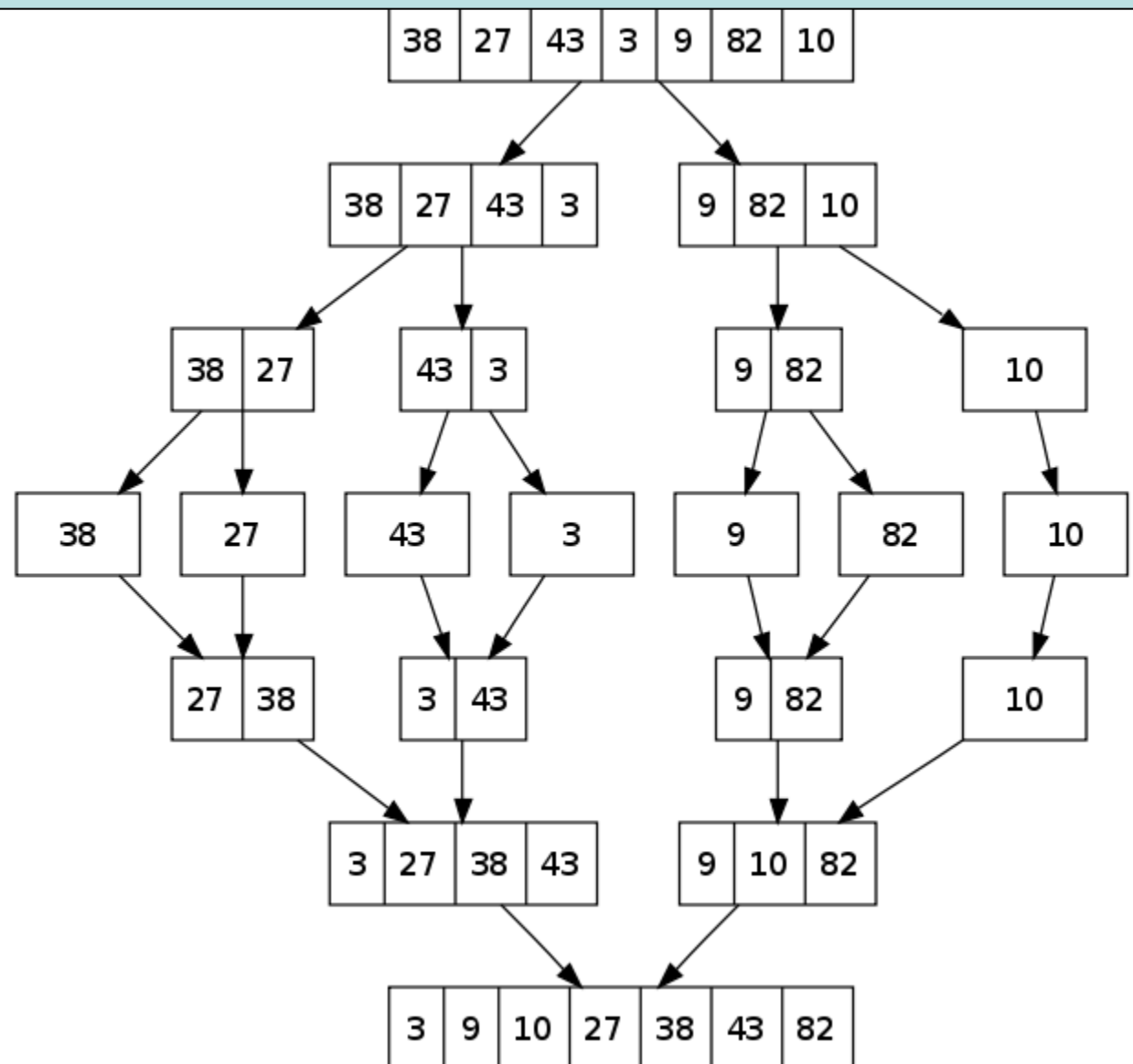
Q: Do we need extra space to merge these two halves together?

A: Unfortunately, yes. Using arrays, we need $O(n)$ auxiliary memory space to merge the lists. Sort a million items, we need another million free slots!

Divide-and-conquer sort, take 2

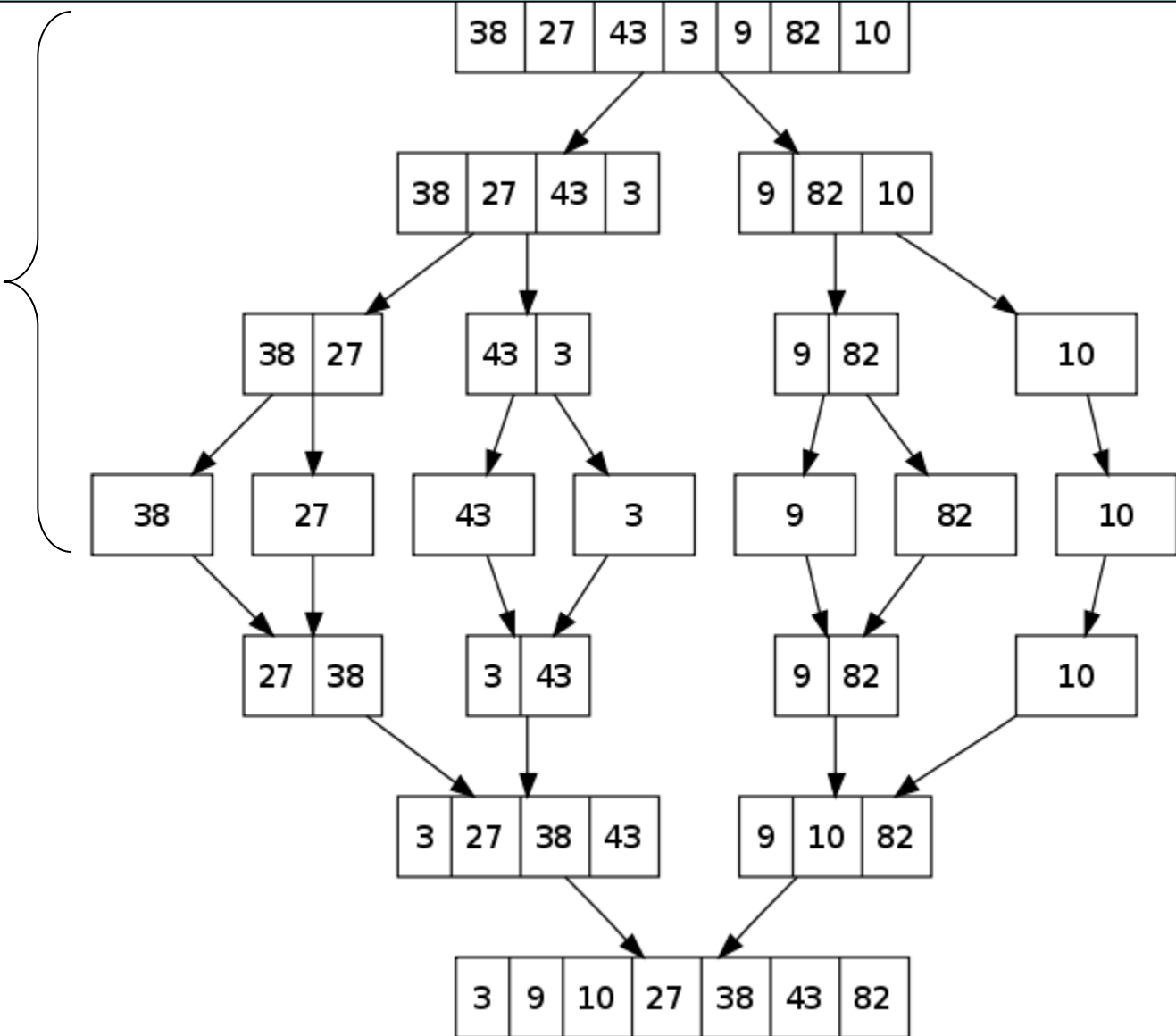
- Merge sort works by splitting the input sequence, recursively sorting these two halves and then merging them back together
- The hard work in merge sort is done by the merge routine – this is where the sorting is really happening...

Divide-and-conquer sort, take 2

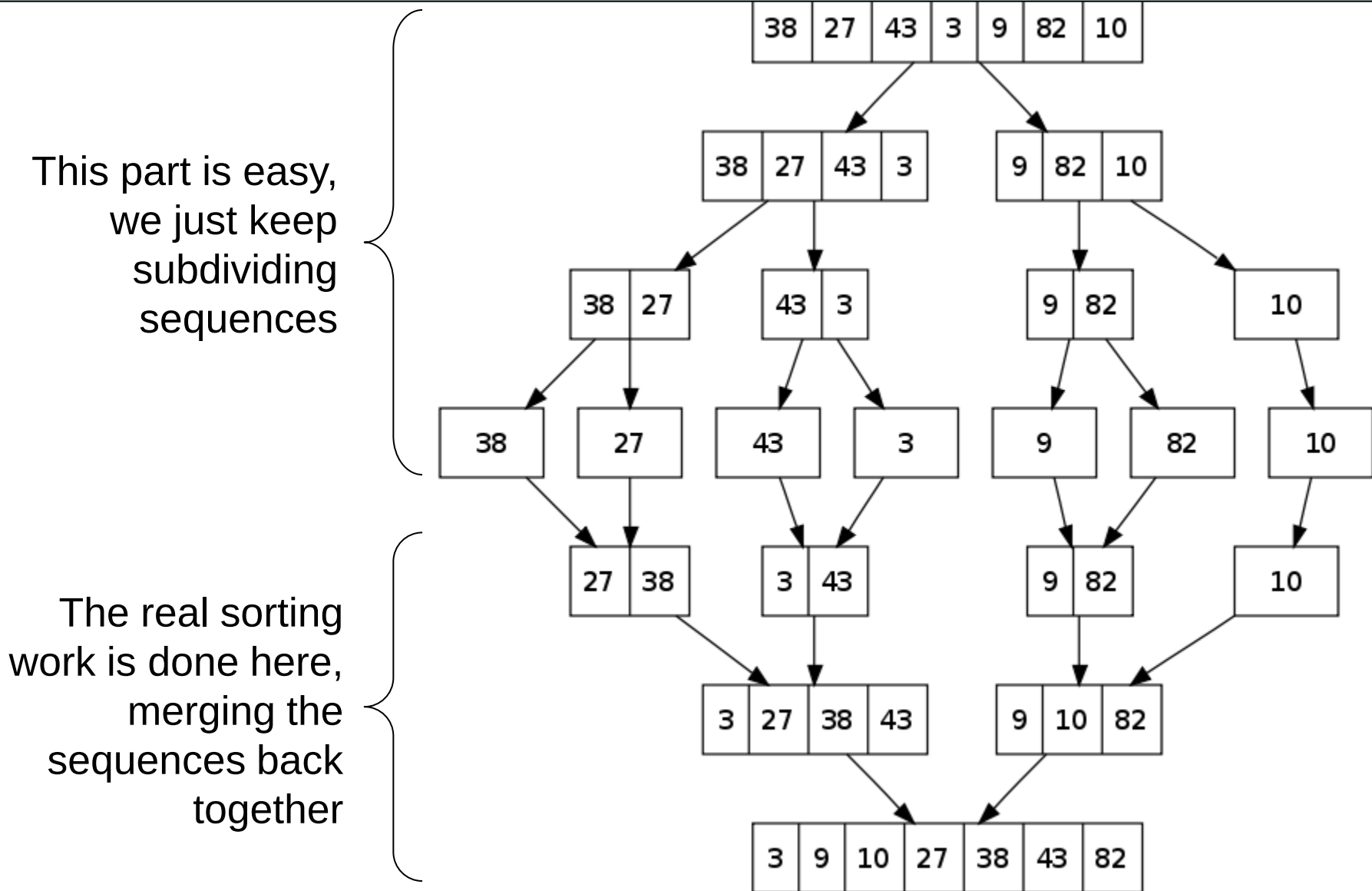


Divide-and-conquer sort, take 2

This part is easy,
we just keep
subdividing
sequences



Divide-and-conquer sort, take 2



Divide-and-conquer sort, take 2

- In other words, we put off the sorting until later, doing the divide step first

How about formulating this the other way round?

- We could do the hard work before the recursive call

Q: How might we state this idea?

Divide-and-conquer sort, take 2

- In other words, we put off the sorting until later, doing the divide step first

How about formulating this the other way round?

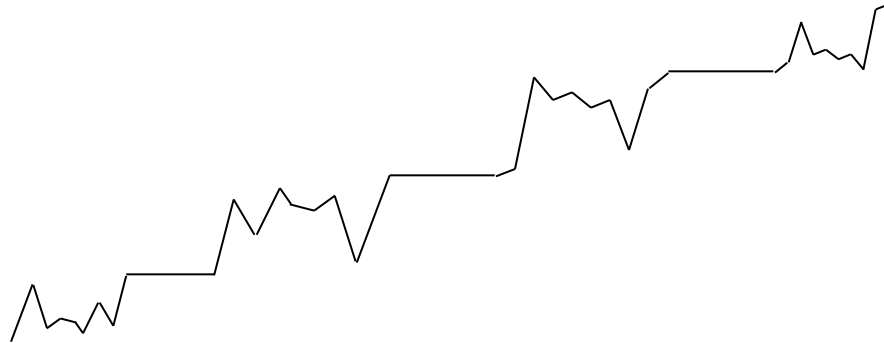
- We could do the hard work before the recursive call

Q: How might we state this idea?

A: A sensible idea seems to be to divide the sequence into those smaller than a certain value and those larger. We can then recursively sort these. Merging will be easy because the two halves will already be in the right order, i.e. we will have done some of the sorting work before the recursive call

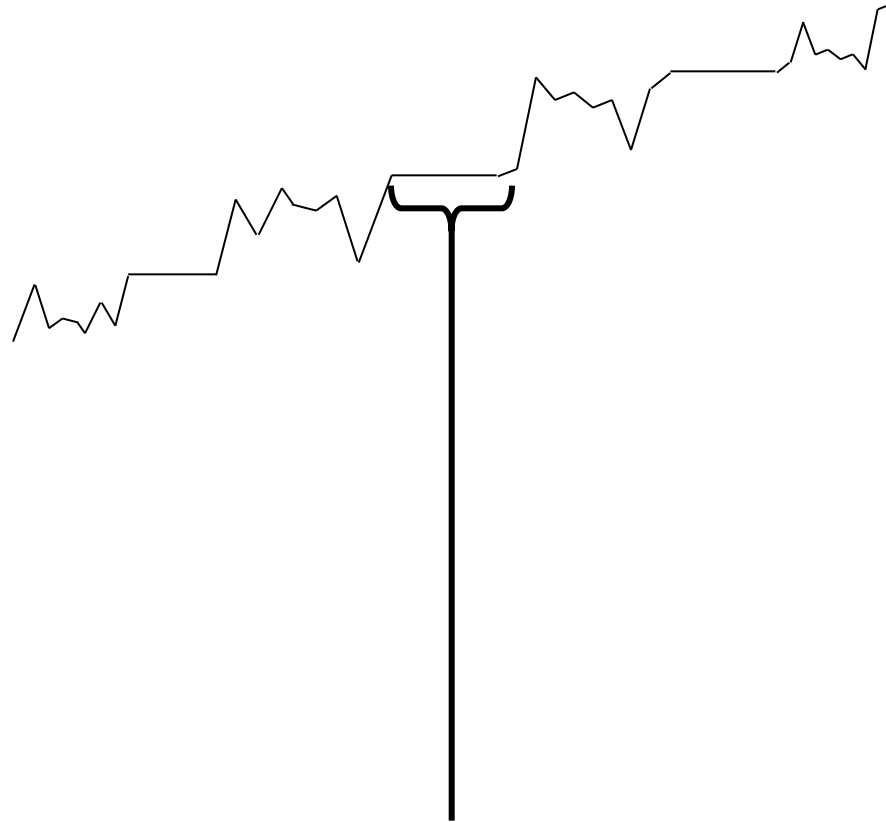
Divide-and-conquer sort, take 2

As a (somewhat confusing) hedgehog diagram, this invariant would look like:



Divide-and-conquer sort, take 2

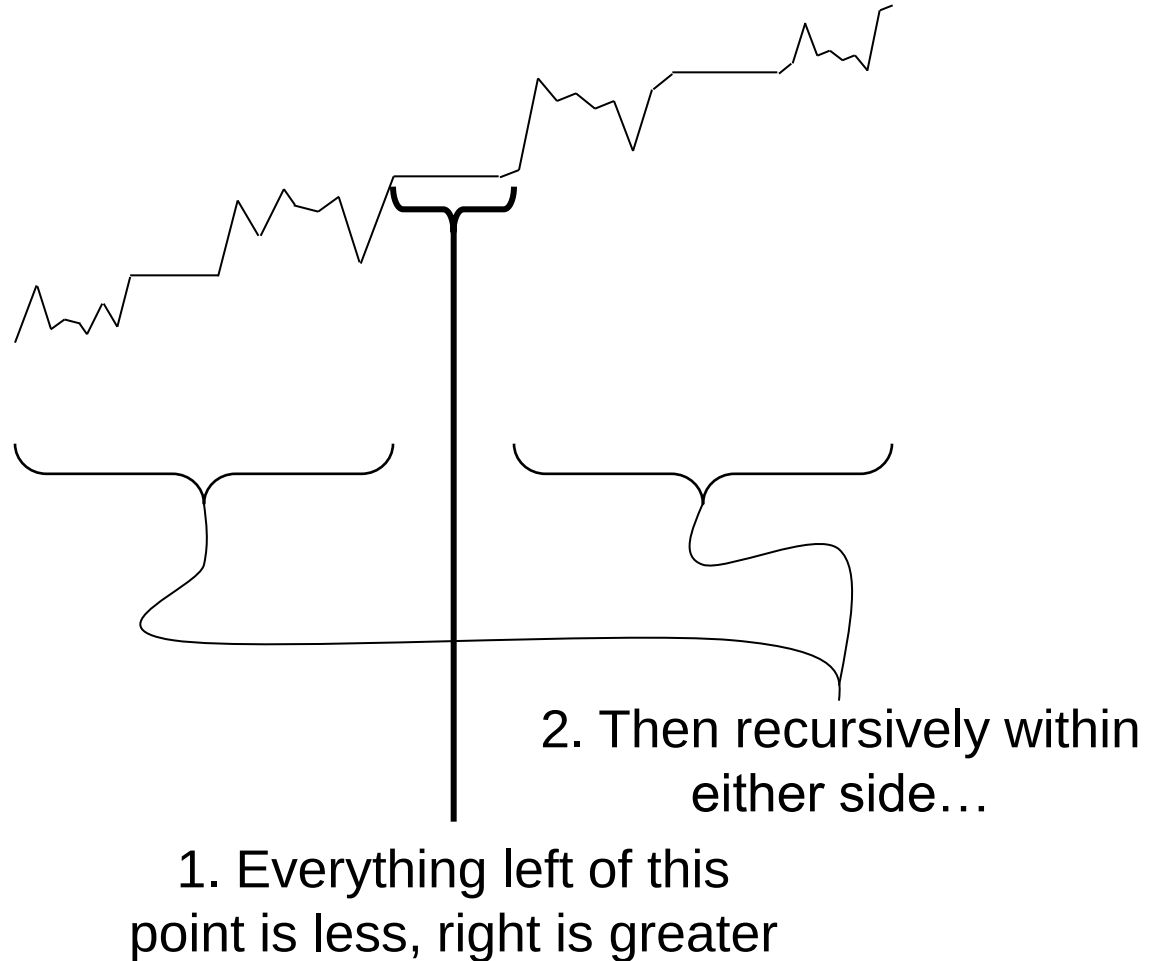
As a (somewhat confusing) hedgehog diagram, this invariant would look like:



1. Everything left of this point is less, right is greater

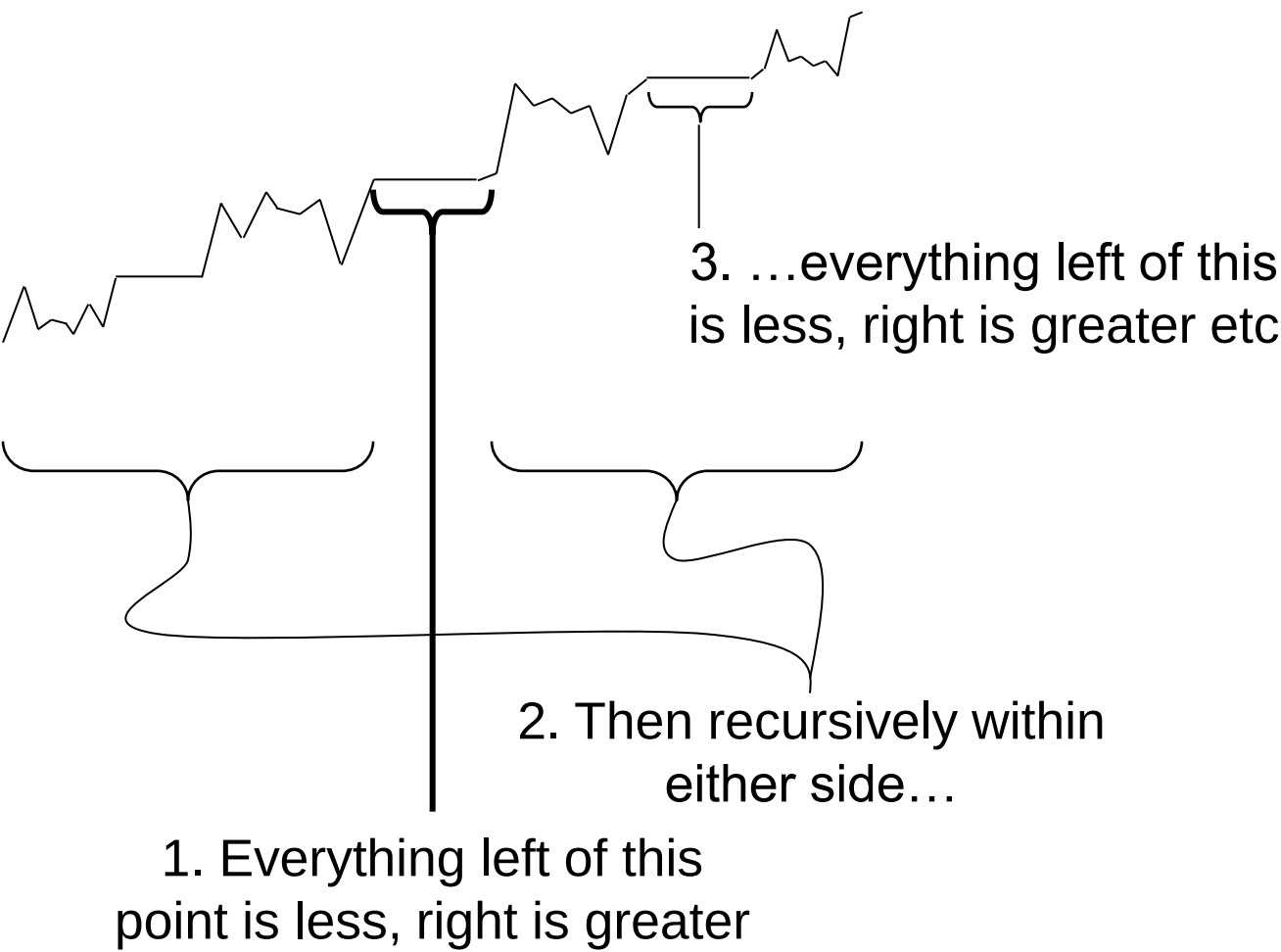
Divide-and-conquer sort, take 2

As a (somewhat confusing) hedgehog diagram, this invariant would look like:



Divide-and-conquer sort, take 2

As a (somewhat confusing) hedgehog diagram, this invariant would look like:



Quicksort

This is exactly the idea behind ***quicksort***

- We start by picking one of the values from the sequence (the ***pivot***)
 - We then arrange all the elements into three groups:
 1. Those that are equal to the pivot
 2. Those that are less than the pivot
 3. Those that are greater than the pivot
- Q:** Is there a way to group items in a sequence into three groups efficiently?
What is the complexity?

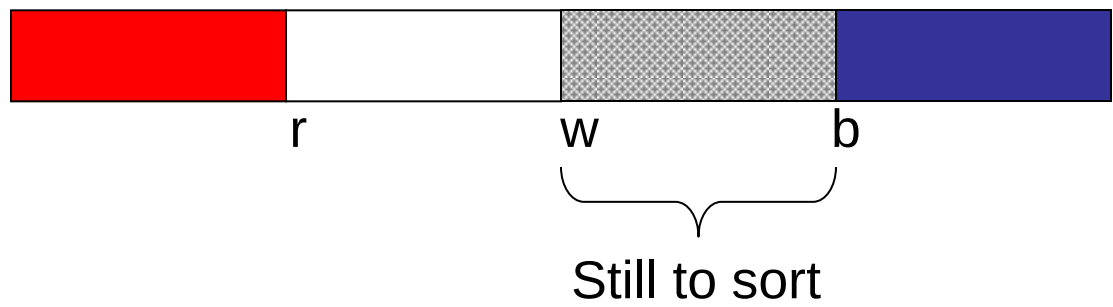
Quicksort

This is exactly the idea behind ***quicksort***

- We start by picking one of the values from the sequence (the ***pivot***)
 - We then arrange all the elements into three groups:
 1. Those that are equal to the pivot
 2. Those that are less than the pivot
 3. Those that are greater than the pivot
- Q:** Is there a way to group items in a sequence into three groups efficiently?
What is the complexity?
- A:** We do not need to compare pairs of items to do this, we can just look at items individually and establish which group they are in. This is exactly the Dutch national flag algorithm we saw last lecture!

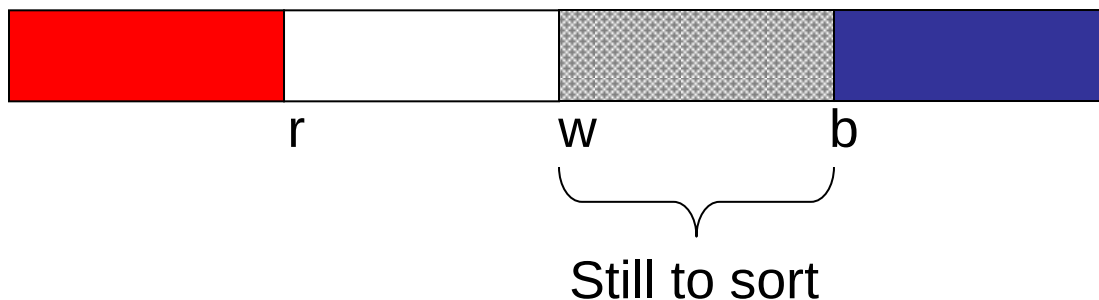
Dutch National Flag for Quicksort

Last time we saw a linear algorithm which solved the Dutch national flag algorithm by maintaining the following invariant:



Dutch National Flag for Quicksort

Last time we saw a linear algorithm which solved the Dutch national flag algorithm by maintaining the following invariant:

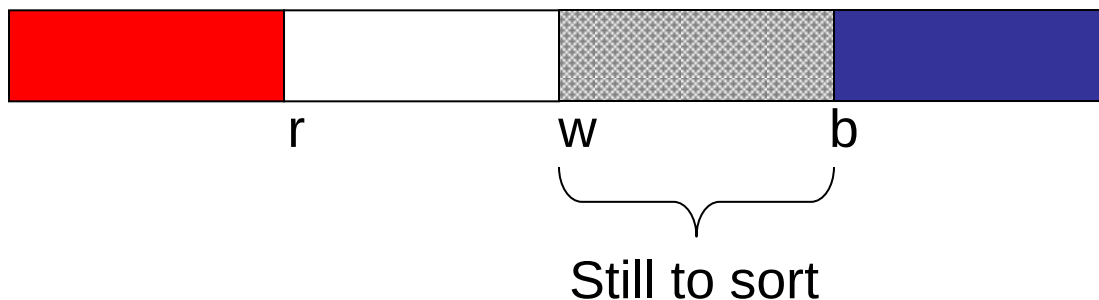


Algorithm sketch for n element sequence:

1. Initialise $r=1$, $w=1$, $b=n+1$
2. Repeatedly perform the following until $w=b$:
 - if $A[w] = \text{red}$, swap($A[r], A[w]$) and increment r and w
 - if $A[w] = \text{white}$, increment w
 - if $A[w] = \text{blue}$, swap($A[w], A[b-1]$), decrement b

Dutch National Flag for Quicksort

Last time we saw a linear algorithm which solved the Dutch national flag algorithm by maintaining the following invariant:



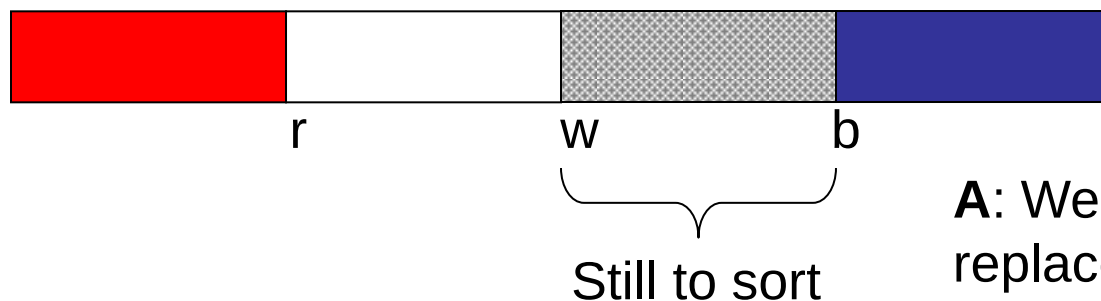
Algorithm sketch for n element sequence:

1. Initialise $r=1$, $w=1$, $b=n+1$
2. Repeatedly perform the following until $w=b$:
 - if $A[w]$ = red, swap($A[r], A[w]$) and increment r and w
 - if $A[w]$ = white, increment w
 - if $A[w]$ = blue, swap($A[w], A[b-1]$), decrement b

Q: What would we need to modify to use this for quicksort?

Dutch National Flag for Quicksort

Last time we saw a linear algorithm which solved the Dutch national flag algorithm by maintaining the following invariant:



A: We can just replace the colour tests with below/above/equal to pivot test

Algorithm sketch for n element sequence:

1. Initialise $r=1$, $w=1$, $b=n+1$

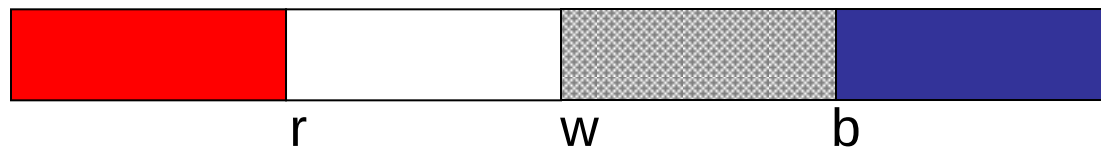
2. Repeatedly perform the following until $w=b$:

if $A[w] = \text{red}$, swap($A[r], A[w]$) and increment r and w
if $A[w] = \text{white}$, increment w
if $A[w] = \text{blue}$, swap($A[w], A[b-1]$), decrement b

Q: What would we need to modify to use this for quicksort?

Dutch National Flag for Quicksort

Last time we saw a linear algorithm which solved the Dutch national flag algorithm by maintaining the following invariant:



Red is $<$ pivot
White is $=$ pivot
Blue is $>$ pivot

Algorithm sketch for n element sequence:

1. Initialise $r=1$, $w=1$, $b=n+1$

2. Repeatedly perform the following until $w=b$:

if $A[w] < \text{pivot}$, swap($A[r], A[w]$) and increment r and w
if $A[w] = \text{pivot}$, increment w
if $A[w] > \text{pivot}$, swap($A[w], A[b-1]$), decrement b

Q: What would we need to modify to use this for quicksort?

Dutch National Flag for Quicksort

- Dutch national flag algorithm lets us *partition* the sequence about the pivot
- Note that the partitioning is done in-place – i.e. we don't need any auxiliary space in which to store elements
- Contrast that to the merge operation in merge sort
- So quicksort looks like it's going to be space efficient
- How about time efficiency?

Quicksort

Function *QuickSort*($A[1..n]$: array of Element)

```
1: if  $n \leq 1$  then  
2:   return  $A$   
3: else  
4:    $pivot := A[n]$   
5:    $(A_1, A_2, A_3) := Partition(A, pivot)$   
6:   invariant  $A_1 < A_2 < A_3$   
7:   return  $concatenate(QuickSort(A_1), A_2, QuickSort(A_3))$   
8: end if
```

Quicksort

Function *QuickSort*($A[1..n]$: array of Element)

```
1: if  $n \leq 1$  then  
2:   return  $A$   
3: else  
4:    $pivot := A[n]$   
5:    $(A_1, A_2, A_3) := Partition(A, pivot)$   
6:   invariant  $A_1 < A_2 < A_3$   
7:   return  $concatenate(QuickSort(A_1), A_2, QuickSort(A_3))$   
8: end if
```

We need to pick a pivot –
for now let's just always
use the last value in the
array

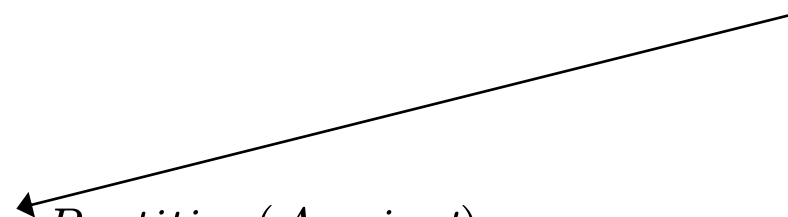


Quicksort

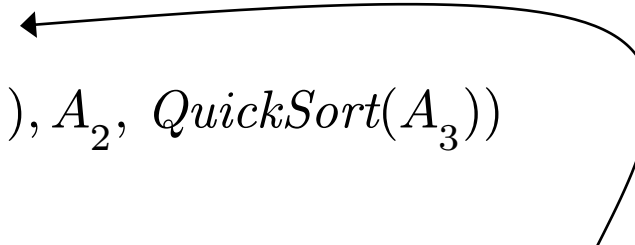
Function *QuickSort*($A[1..n]$: array of Element)

```
1: if  $n \leq 1$  then  
2:   return  $A$   
3: else  
4:    $pivot := A[n]$   
5:    $(A_1, A_2, A_3) := Partition(A, pivot)$   
6:   invariant  $A_1 < A_2 < A_3$   
7:   return  $concatenate(QuickSort(A_1), A_2, QuickSort(A_3))$   
8: end if
```

We need to pick a pivot –
for now let's just always
use the last value in the
array



The partition function just applies
the DNF algorithm. Values in A_1
are less than pivot, A_2 equal and
 A_3 greater.



Quicksort

Function *QuickSort*($A[1..n]$: array of Element)

```
1: if  $n \leq 1$  then  
2:   return  $A$   
3: else  
4:    $pivot := A[n]$   
5:    $(A_1, A_2, A_3) := Partition(A, pivot)$   
6:   invariant  $A_1 < A_2 < A_3$   
7:   return  $concatenate(QuickSort(A_1), A_2, QuickSort(A_3))$   
8: end if
```

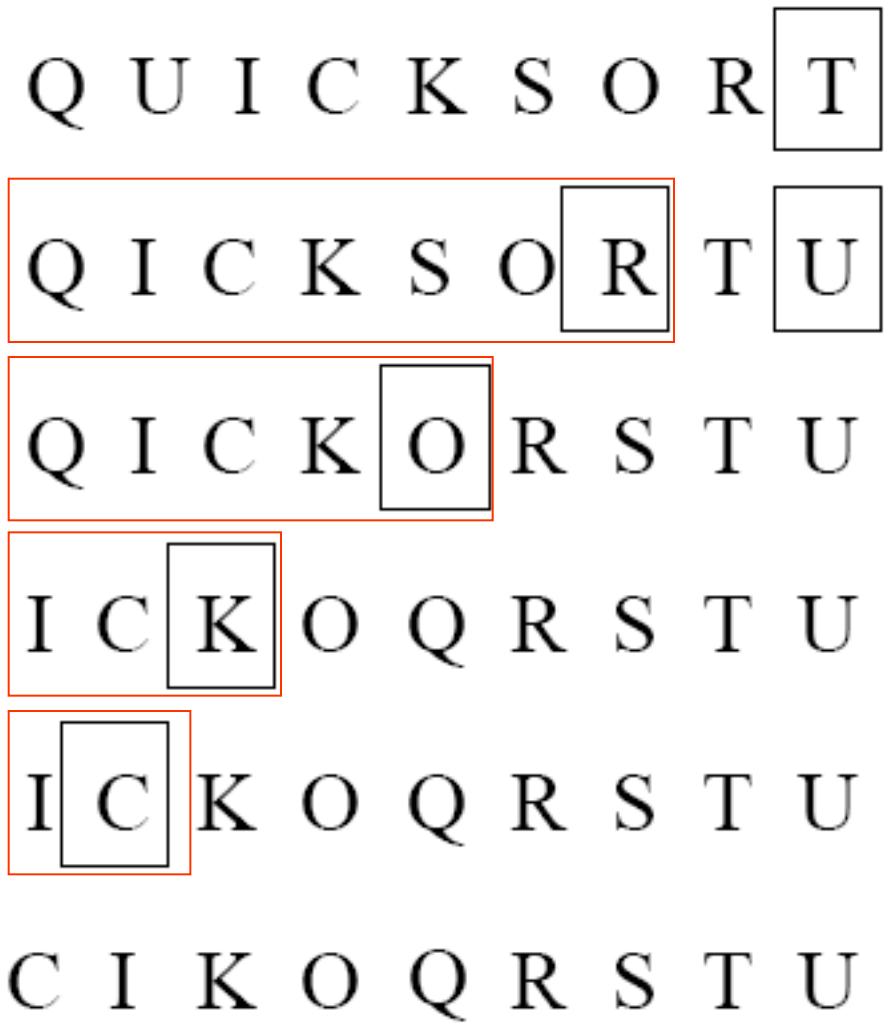
We need to pick a pivot –
for now let's just always
use the last value in the
array

The partition function just applies
the DNF algorithm. Values in A_1
are less than pivot, A_2 equal and
 A_3 greater.

We then return the concatenation of
quicksorting the bottom, values equal
to the pivot and quicksorting the top

Quicksort Example

Boxed elements are current pivots, red boxes are currently unsorted sequences



Quicksort – Choosing a pivot

- The choice of pivot has a dramatic effect on the efficiency of quicksort
- If we always choose the best pivot the complexity is $O(n \log n)$, if we always choose the worst it is $O(n^2)$
- After partitioning, the pivot is in its correct position in the final sorted list
- The pivot must be one of the elements from the sequence
- This is how we make progress – the pivot is now the right place
- So the questions are:
 1. What is a good choice of pivot?
 2. What is a bad choice of pivot?
 3. Can we try and improve our chances of picking a good pivot?
 4. If so, how likely are we to pick a bad pivot?

Asymptotic Analysis of Quicksort

- We have focussed on worst case analysis so far
- If we can make the worst case highly improbable, we are justified in considering the average case
- This is the case with quicksort, which turns out to be the most efficient sorting algorithm in practice
- Let's consider the worst case first...

Asymptotic Analysis of Quicksort

- In our pseudocode above, we always picked the last value in the sequence as the pivot

Q: For what sort of inputs would this be a bad choice of pivot?

Asymptotic Analysis of Quicksort

- In our pseudocode above, we always picked the last value in the sequence as the pivot

Q: For what sort of inputs would this be a bad choice of pivot?

A: Sorted or reverse sorted. In this case, everything will fall on one side of the pivot. i.e.:

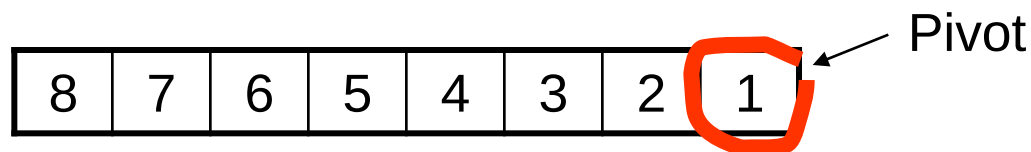
8	7	6	5	4	3	2	1
---	---	---	---	---	---	---	---

Asymptotic Analysis of Quicksort

- In our pseudocode above, we always picked the last value in the sequence as the pivot

Q: For what sort of inputs would this be a bad choice of pivot?

A: Sorted or reverse sorted. In this case, everything will fall on one side of the pivot. i.e.:

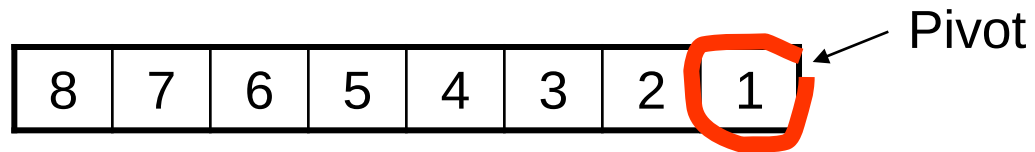


Asymptotic Analysis of Quicksort

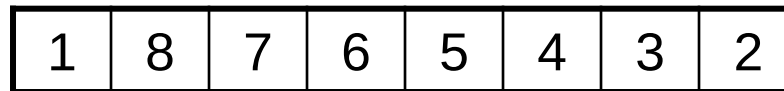
- In our pseudocode above, we always picked the last value in the sequence as the pivot

Q: For what sort of inputs would this be a bad choice of pivot?

A: Sorted or reverse sorted. In this case, everything will fall on one side of the pivot. i.e.:



After partitioning:

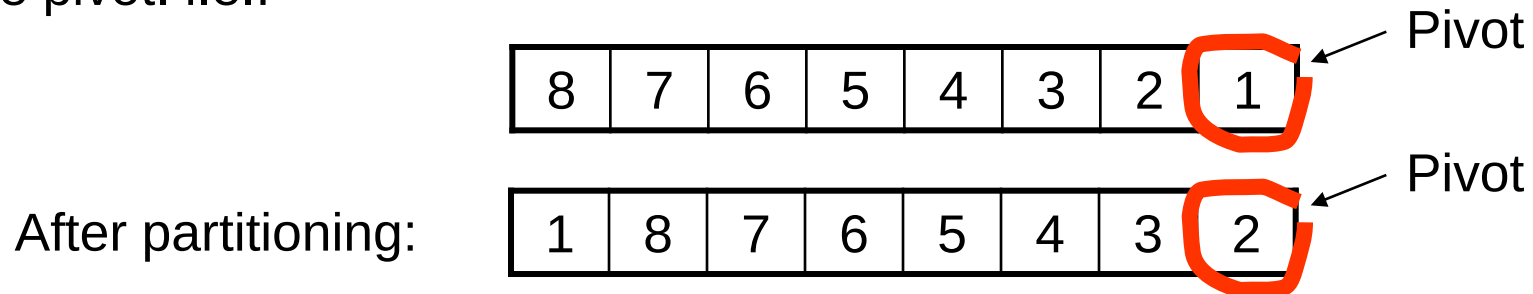


Asymptotic Analysis of Quicksort

- In our pseudocode above, we always picked the last value in the sequence as the pivot

Q: For what sort of inputs would this be a bad choice of pivot?

A: Sorted or reverse sorted. In this case, everything will fall on one side of the pivot. i.e.:

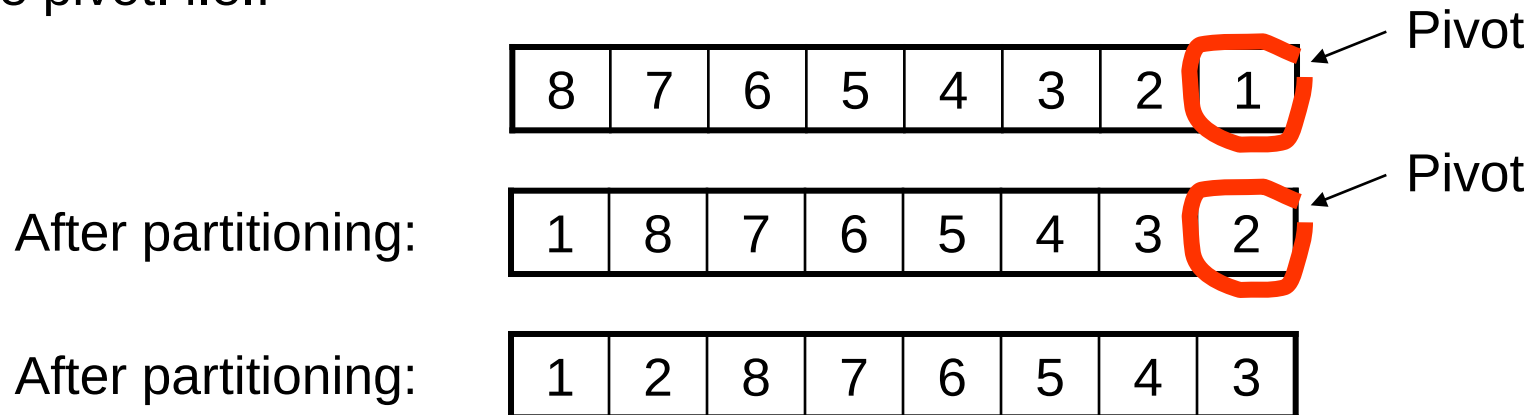


Asymptotic Analysis of Quicksort

- In our pseudocode above, we always picked the last value in the sequence as the pivot

Q: For what sort of inputs would this be a bad choice of pivot?

A: Sorted or reverse sorted. In this case, everything will fall on one side of the pivot. i.e.:

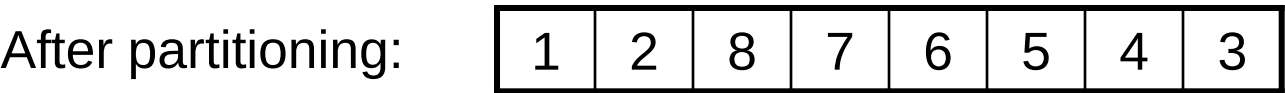
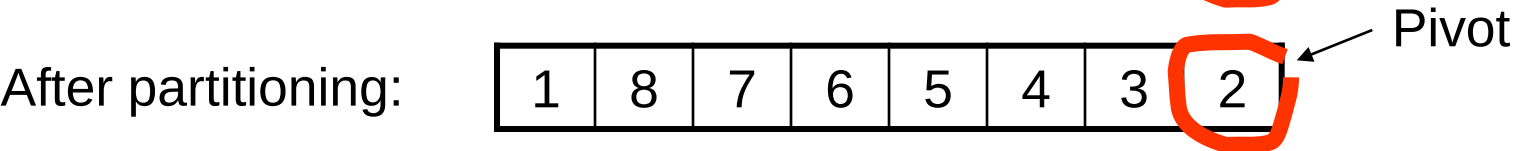
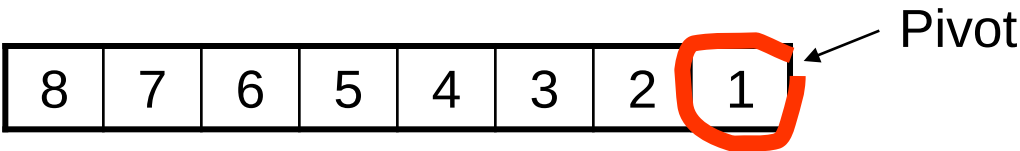


Asymptotic Analysis of Quicksort

▪ In our pseudocode above, we always picked the last value in the sequence as the pivot

Q: For what sort of inputs would this be a bad choice of pivot?

A: Sorted or reverse sorted. In this case, everything will fall on one side of the pivot. i.e.:



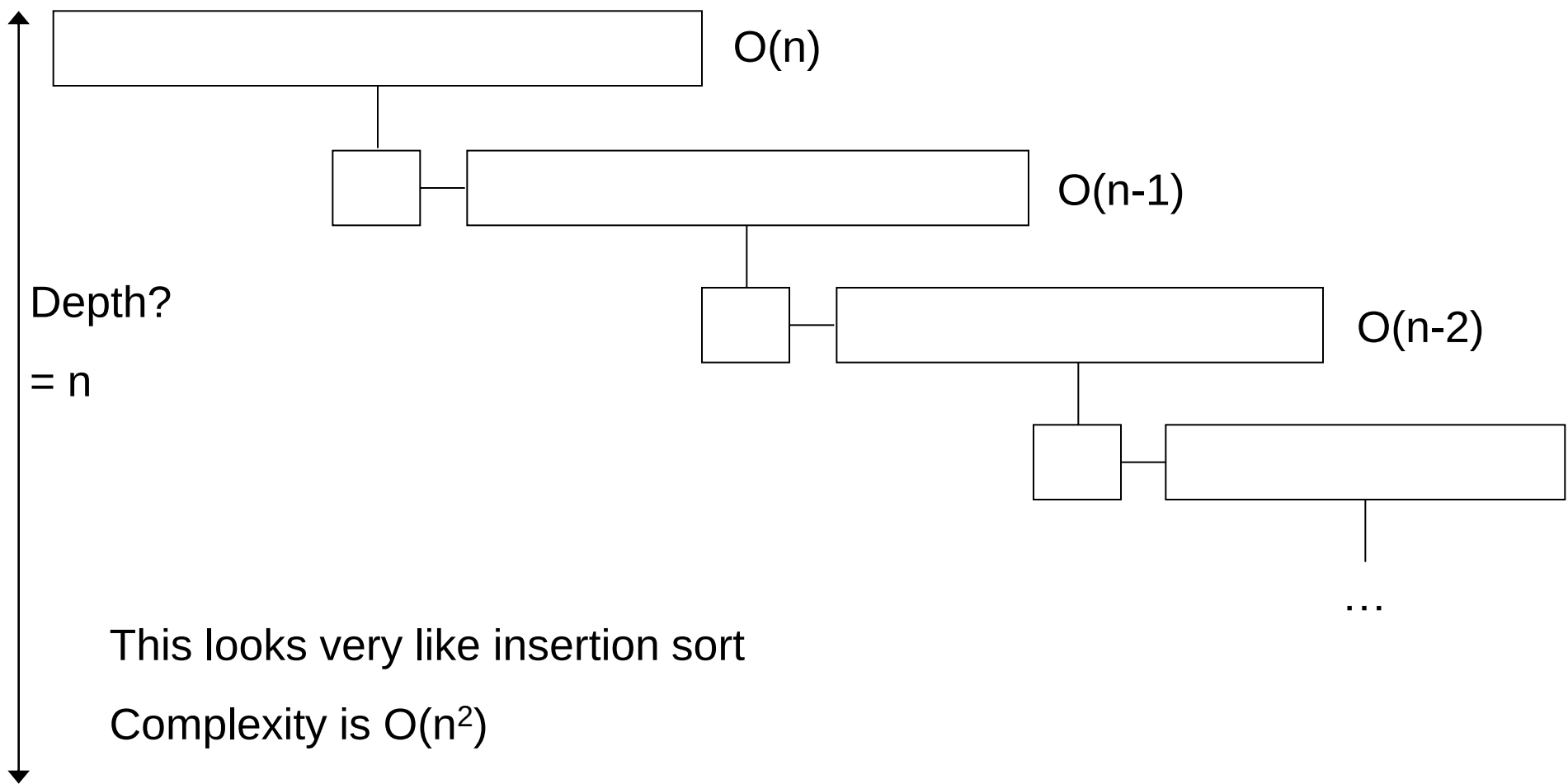
And so on...

Partitioning costs us $O(n)$ work and we can see we're going to have to do this n times giving $O(n^2)$ complexity

Asymptotic Analysis of Quicksort

Worst case as a recursion tree

Work per level is the partitioning operation:



Asymptotic Analysis of Quicksort

Q: What would be the best choice of pivot?

Asymptotic Analysis of Quicksort

Q: What would be the best choice of pivot?

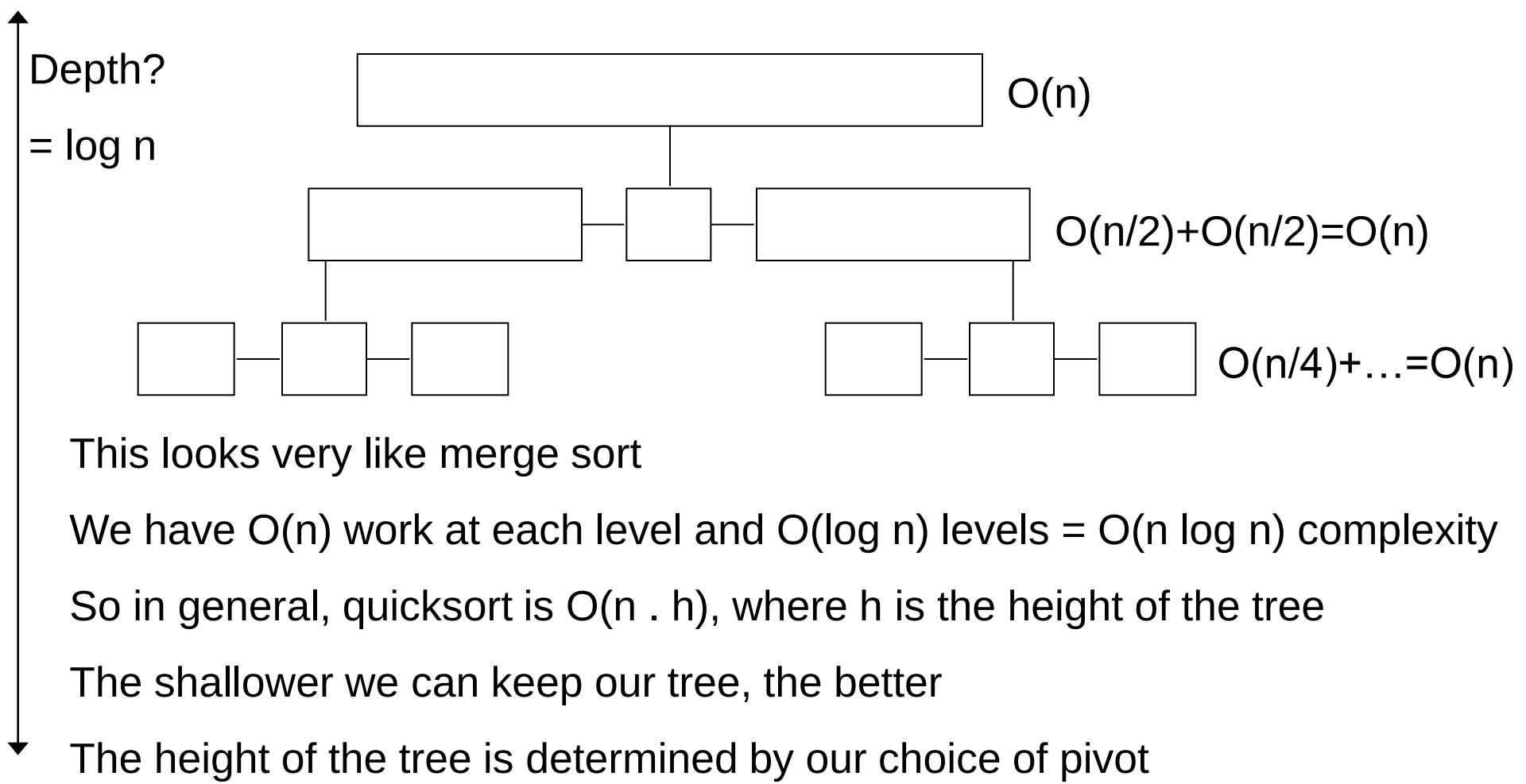
A: The median element

- In this case, the length of the sequence will always be halved
- As we know, this will give us a tree of depth $\log n$...

Asymptotic Analysis of Quicksort

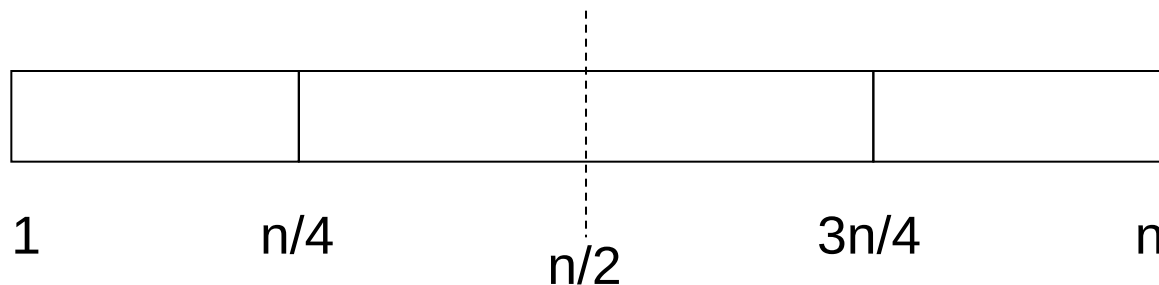
Best case as a recursion tree

Work per level is the partitioning operation:



Asymptotic Analysis of Quicksort

- If we get really lucky and always pick the median as our pivot, we'll get $O(n \log n)$ performance
- If we get really unlucky and always pick the smallest or largest element as our pivot we'll get $O(n^2)$ performance
- The question is, how likely are we to pick a “good” or “bad” pivot?
- Intuition:

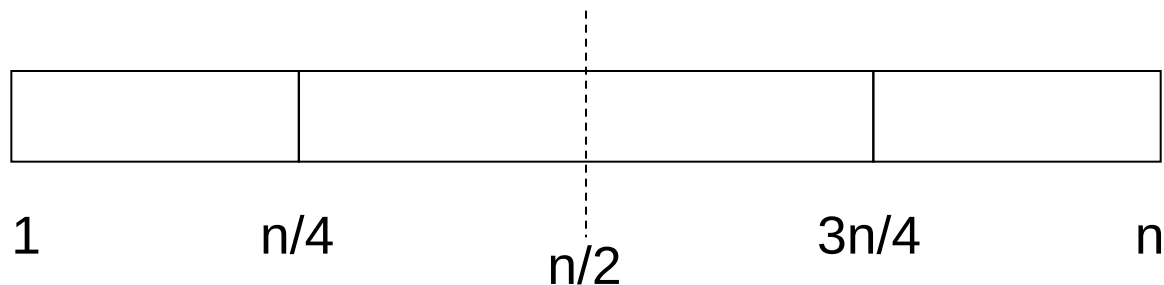


Let's say that a pivot between $n/4$ and $3n/4$ is “good”

Q: How likely are we to pick a “good” pivot?

Asymptotic Analysis of Quicksort

- If we get really lucky and always pick the median as our pivot, we'll get $O(n \log n)$ performance
- If we get really unlucky and always pick the smallest or largest element as our pivot we'll get $O(n^2)$ performance
- The question is, how likely are we to pick a “good” or “bad” pivot?
- Intuition:

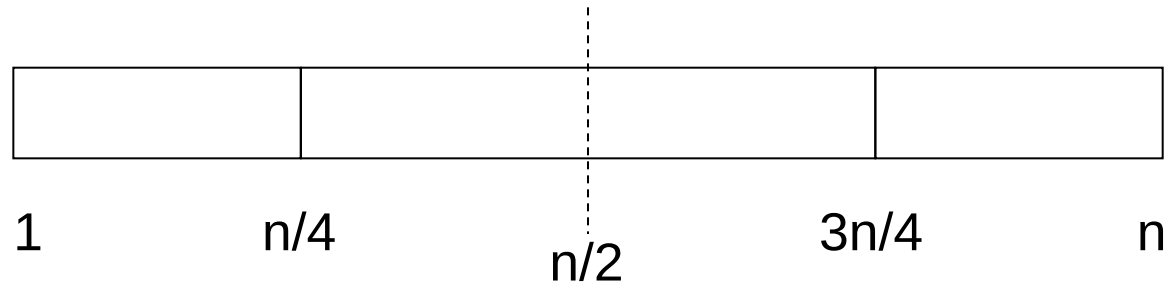


Let's say that a pivot between $n/4$ and $3n/4$ is “good”

Q: How likely are we to pick a “good” pivot?

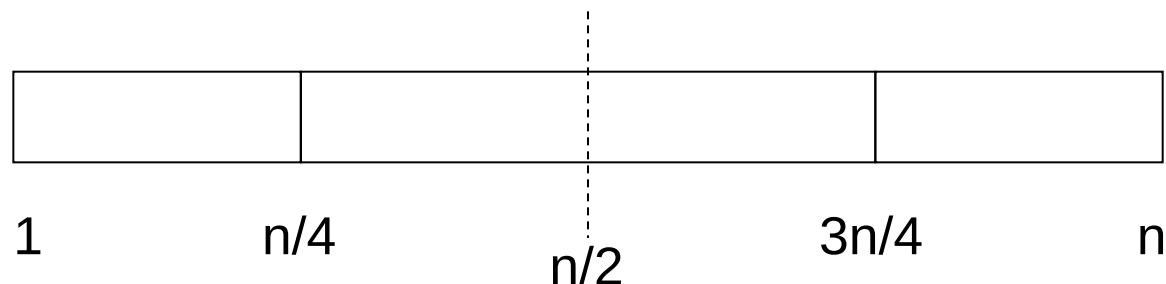
A: Probability = 0.5

Asymptotic Analysis of Quicksort



Q: What's the worst “good” pivot?

Asymptotic Analysis of Quicksort



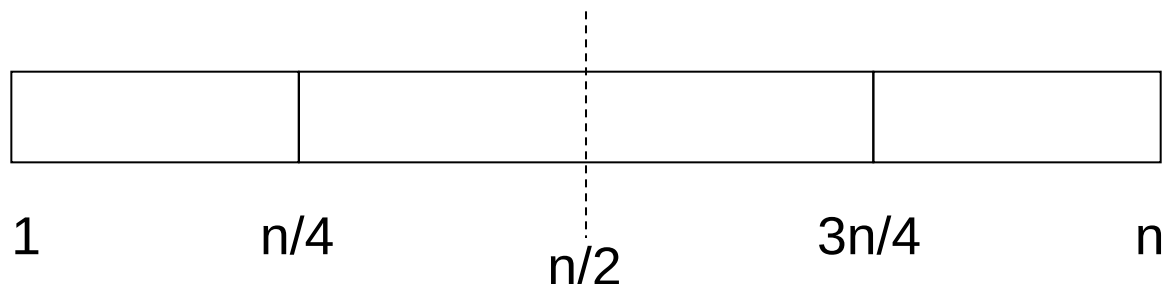
Q: What's the worst “good” pivot?

A: One that leaves us with $3/4$ of the sequence on one side every time
The depth of the bottom of this tree, h_g , will be how many times we have to multiply n by $3/4$ until we get to 1:

$$(3/4)^{h_g} n = 1$$

$$h_g = \log_{4/3} n$$

Asymptotic Analysis of Quicksort



Q: What's the worst “good” pivot?

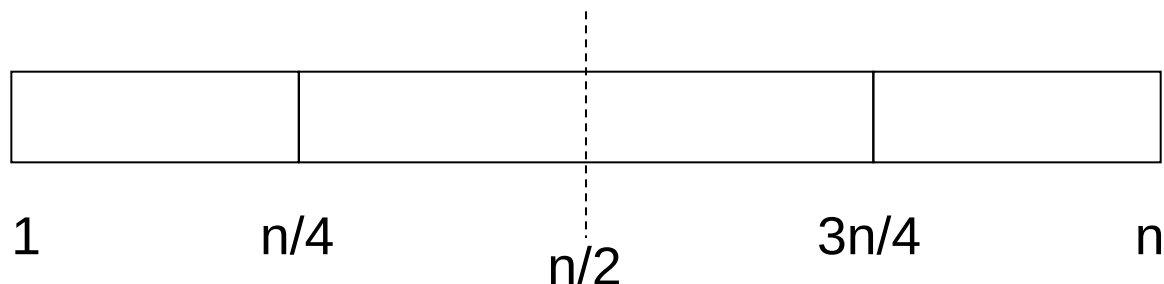
A: One that leaves us with $3/4$ of the sequence on one side every time
The depth of the bottom of this tree, h_g , will be how many times we have to multiply n by $3/4$ until we get to 1:

$$\begin{aligned}(3/4)^{h_g} n &= 1 \\ h_g &= \log_{4/3} n\end{aligned}$$

So if we always picked a “good” pivot, the tree would be $O(\log n)$ deep

But we will pick a “bad” pivot with probability 0.5

Asymptotic Analysis of Quicksort



Let's assume everytime we pick a bad pivot, we pick the worst possible one, i.e. the maximum or minimum

On these splits, let's say approximately we achieve nothing (i.e. don't reduce the size of the sequence)

The deepest path through a randomly constructed quicksort tree will pass through equal numbers of good and bad splits

Hence, the bad splits will just double the depth (constant factor):

$$h \approx 2h_g = 2 \log_{4/3}(n) = \Theta(\log n)$$

Randomised Quicksort

Of our quicksort implementation above, we can now say:

“Quicksort runs in $\Theta(n \log n)$ time with high probability, if you give me a randomised input”

But we can effectively achieve a randomised input by making our pivot selection random. This would allow us to say:

“Randomised quicksort runs in $\Theta(n \log n)$ time on any input with high probability”

Even if we were given a sorted input, half the time we would pick “good” pivots (unless we were particularly unlucky) giving us an $O(\log n)$ tree depth

Quicksort Summary

This randomised quicksort is the best choice for sorting large arrays in practice

This is because it has a much smaller constant factor than merge sort

Final notes on quicksort:

1. It works in-place, no extra space needed
2. Another strategy for picking the pivot is to look at the first, middle and last values of the sequence and use the median of these three as the pivot (hope to approximate median of whole sequence – good for partially ordered input)
3. As with merge sort, switch to insertion sort when sequence size get small

Shellsort

A final sorting algorithm: ***shellsort***

Intended to be a divide-and-conquer version of insertion sort (but isn't)

The idea is to divide the sequence into k interleaved subsequences and insertion sort these separately

We then reduce k and repeat until k reaches 1

E.g. $k=4$:



Elements with the same colour would form a subsequence to be insertion sorted

Analysing shellsort is extremely difficult, by choosing different sequences for k , different worst case complexities arise. Best known sequence gives: $\Theta(n^{8/7})$

- Better than quadratic, but still polynomial so dominates $O(n \log n)$

Shellsort

Shellsort works by getting things into roughly the right position at one level before insertion sorting at the next level

It works in place and the code is extremely small

Possibly useful in applications with little memory where the sorting problem isn't too large

Stable Sorting

We've been classifying sorting algorithms by their time and space complexity

There is another classification that is useful...

Stable Sorting

A sorting algorithm is said to be ***stable*** if elements with equal keys retain their relative order. E.g.:

Unsorted input:

Data

Key

A	B	C	D	E	F
6	19	4	6	3	1

Stable Sorting

A sorting algorithm is said to be **stable** if elements with equal keys retain their relative order. E.g.:

Unsorted input:	Data	A	B	C	D	E	F
	Key	6	19	4	6	3	1

Sorted output 1:	F	E	C	A	D	B
	1	3	4	6	6	19

Sorted output 2:	F	E	C	D	A	B
	1	3	4	6	6	19

Stable Sorting

A sorting algorithm is said to be **stable** if elements with equal keys retain their relative order. E.g.:

Unsorted input:	Data	A	B	C	D	E	F
	Key	6	19	4	6	3	1

Sorted output 1:	F	E	C	A	D	B
	1	3	4	6	6	19

Sorted output 2:	F	E	C	D	A	B
	1	3	4	6	6	19

Stable

Stable Sorting

A sorting algorithm is said to be **stable** if elements with equal keys retain their relative order. E.g.:

Unsorted input:

Data	A	B	C	D	E	F
Key	6	19	4	6	3	1

Sorted output 1:

F	E	C	A	D	B
1	3	4	6	6	19

Stable

Sorted output 2:

F	E	C	D	A	B
1	3	4	6	6	19

Not stable

Stable Sorting

A sorting algorithm is said to be **stable** if elements with equal keys retain their relative order. E.g.:

Unsorted input:

Data

A	B	C	D	E	F
6	19	4	6	3	1

Key

Sorted output 1:

F	E	C	A	D	B
1	3	4	6	6	19

Stable

Sorted output 2:

F	E	C	D	A	B
1	3	4	6	6	19

Not stable

Q: Why might a stable sorting algorithm be useful?

Stable Sorting

A: It preserves some information about the state of the sequence before sorting

- The best example of why this is useful is when we wish to sort by secondary keys, e.g. sort students by exam marks and secondary sort them by surname:

6	Smithers
5	Pimpernel
7	Fotheringham
4	Gates
9	Newton
9	Einstein
6	Page
6	Jobs

If we sort by surname first and then apply a stable sort on exam mark, we will preserve the surname ordering for equal marked students...

Stable Sorting

Sorted by surname:

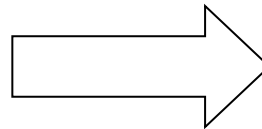
9	Einstein
7	Fotheringham
4	Gates
6	Jobs
9	Newton
6	Page
5	Pimpernel
6	Smithers

Stable Sorting

Sorted by surname:

9	Einstein
7	Fotheringham
4	Gates
6	Jobs
9	Newton
6	Page
5	Pimpernel
6	Smithers

Stable sort by exam mark:



Desired result:

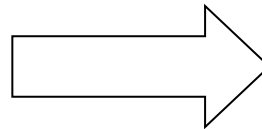
4	Gates
5	Pimpernel
6	Jobs
6	Page
6	Smithers
7	Fotheringham
9	Einstein
9	Newton

Stable Sorting

Sorted by surname:

9	Einstein
7	Fotheringham
4	Gates
6	Jobs
9	Newton
6	Page
5	Pimpernel
6	Smithers

Stable sort by exam mark:



Desired result:

4	Gates
5	Pimpernel
6	Jobs
6	Page
6	Smithers
7	Fotheringham
9	Einstein
9	Newton

Exercise: Are any/all of the sorting algorithms we've looked at stable?

Sorting Contiguous versus Sorting Linked Data Structures

- All the algorithms we've seen can be implemented to work with either arrays or linked lists
- Swapping pointers is generally faster than swapping records, especially if the records are large
- There is a special version of merge sort for linked lists which works in place (no auxiliary memory required)

Lessons of Sorting

We can take everything we know about sorting to arrive at some rules of thumb:

1. Quadratic sorters (bubblesort, selection sort, insertion sort) are slow for large input, but insertion sort is best
2. Because of its small constant factor, insertion sort beats asymptotically better algorithms for small inputs – so use insertion sort for inputs less than around 30 items
3. For anything but small sequences, use quicksort for arrays and merge sort for linked lists. Switch to insertion sort when recursion has reached list of around 30 items (exact number depends on implementation of both algorithms)
4. Shell sort may be a good choice for moderately sized input where limited memory is available

Selection Problems

Selection problems can easily be solved using sorting, but do not need the full power of sorting

E.g.: find me the biggest/smallest/median elements, return me the k th smallest element

The easy way to answer these problems is to sort the sequence and simply pick them out from the sorted sequence

Q: Can we do any better?

Q: Suggest an algorithm for biggest/smallest. Complexity?

Q: Suggestions for median? Seems a lot harder – can you use quicksort as inspiration?

Conclusions

We should now be able to choose an appropriate sorting algorithm for our application and know what the complexity will be

Next time:

- Sorted sequences, tree-based dictionaries

Recommended reading for this lecture:

- Mehlhorn Chapter 5, 5.4 onwards
- Cormen Chapters 7 and 8
- Skiena 4.6 and 4.7